

Software Engineering

Rapport détaillé

1- Initialisation de l'architecture du projet et compréhension du projet

La première étape du projet a consisté à analyser et comprendre le code source destiné à la mise en production. Au début, chaque membre de l'équipe a essayé de comprendre le script du projet Titanic localement, sans structure spécifique, afin de saisir la logique et le code complet.

2- Mise en Place de l'Architecture avec Cookiecutter

Une fois le code compris, nous avons pris le template Cookiecutter Data Science pour définir notre environnement de travail.

Nous avons installé l'outil via pip et généré l'arborescence standard. Cependant, pour répondre aux besoins de notre exécution "fichier par fichier", nous avons personnalisé le template en supprimant les composants non utiles (comme plots.py ou certains sous-dossiers de data et même main.py).

En revanche, l'architecture a été adaptée pour inclure un dossier `titanic_project` pour le code source et un dossier `test` pour les tests unitaires

3- Refactorisation

Après avoir mis en place cette structure, nous avons commencé la refactorisation. Le code, initialement présent sous forme de cellules Jupyter, a été divisé en trois modules dans le dossier `titanic_project` :

1. `data_preprocessing.py` : Responsable du chargement des fichiers CSV et de la préparation des données, avec une sauvegarde dans `data/interim`.
2. `model_evaluation.py` : Pour les calculs des statistiques de survie entre les hommes et les femmes. Ces calculs sont stockés dans le dossier `data/processed`.

3. `model_training.py` : Consacré à l'entraînement du modèle avec le package `sklearn` et à la génération des prédictions. Ce programme renvoie un `csv` dans lequel on a les survivant en 1, le fichier est sauvegardé dans `data/reports`.

Chaque script se lance automatiquement, il n'y a aucun changement à faire et nous avons appliqué la mise en forme PEP8 avec `Flake8`, `Isort` et `Black`, afin que notre code respecte ce style de code.

Pour ce qui est des librairies, nous avons mis celles qui sont essentielles dans `requirements.txt`.

Enfin certaines variables qui se nommaient `x` ou `y` ont été renommées en nom de variables explicites.

4- Mise en place des tests unitaires

Ensuite, pour garantir la qualité du code, nous avons transformé les programmes en fonctions. Cela a permis d'implémenter des tests unitaires avec la bibliothèque `pytest`.

Chaque module Python possède son équivalent dans le dossier `tests`, permettant de valider individuellement chaque étape du pipeline (chargement, calculs et prédictions).

5- Ajouter de la documentation

Pour chacun de nos scripts nous avons ajouté des commentaires afin de préciser ce que nous faisons.

Nous avons de plus créé un fichier `ReadMe` afin de donner davantage de contexte à notre projet et que chaque "lecteur" puisse le comprendre.

6- Mettre en Place une Pipeline CI/CD

La pipeline est configurée pour se déclencher automatiquement à chaque `push` ou `pull request` effectué sur la branche principale du projet (`master`).

Cette configuration est définie dans le fichier `YAML` via la section `on`, qui spécifie les événements surveillés par `GitHub Actions`.

Le workflow s'exécute sur une machine virtuelle utilisant le système d'exploitation `Ubuntu (ubuntu-latest)`.

Ce choix s'explique par sa compatibilité avec `Python` ainsi que par sa large utilisation dans les environnements `CI/CD`, ce qui garantit une exécution stable et reproductible.

La pipeline installe `Python 3.13`, puis l'ensemble des dépendances nécessaires au projet à partir du fichier `requirements.txt`.

Cette étape permet de reproduire fidèlement l'environnement de développement local lors de l'exécution automatique du workflow.

L'outil Black est utilisé afin de vérifier le respect des règles de formatage du code source.

Si le code ne respecte pas le format attendu, la pipeline échoue, obligeant ainsi les développeurs à corriger les problèmes avant toute intégration sur la branche principale.

L'outil Flake8 permet d'analyser le code afin de détecter différentes erreurs courantes, telles que :

- une mauvaise indentation
- des lignes trop longues
- des imports inutilisés
- des incohérences avec les bonnes pratiques Python

Cette étape contribue à maintenir un code lisible, cohérent et conforme aux standards de développement.

Lors de la mise en place de la pipeline on a constaté des erreurs :

- Nous avons un problème avec le formatage Black pour certains formats. On a relancé Black sur nos fichiers en erreur sur VSCode et on les a push sur Github
- Nos fichiers tests n'arrivaient pas à importer les modules. Nous les avons changé de fichier mais avons oublié de repréciser le bon fichier.
- Un de nos tests étaient mal paramétré et donc forcer l'erreur de la CI, nous l'avons donc remplacer

Enfin, les tests unitaires sont exécutés à l'aide de Pytest.

Cette étape permet de vérifier le bon fonctionnement des différentes fonctionnalités du projet et de s'assurer qu'aucune modification récente n'introduit de régression.

L'organisation

Nous avons push notre cookiecutter, avec la refactorisation du programme déjà faite, sur notre projet GitHub, sur lequel nous étions tous contributeurs. Chacun avait des tâches à faire qu'il réalisait sur VSCode en ayant au préalable récupéré l'arborescence via GitHub, et push par la suite chaque avancée.